Design Practices in Computer Science: CS-3810 Final Project Report

Interactive Delhi Metro Route Planner and Visualiser

Department of Computer Science, Ashoka University

1 Project Identification

Team Name	Connect
Team Members	Dhivya, Saraswathi
Roll Numbers	1020241642, 1020231012
Assigned Project	Interactive Delhi Metro Route Planner and Visualiser
Repository Link	https://github.com/Ashoka-CS-3810-Design-Practices-in-CS/project-1-connect

2 Assigned Project

Project Title: Interactive Delhi Metro Route Planner and Visualiser.
Problem statement: Delhi Metro riders need a quick way to find the fastest route between two stations, along with the interchanges, without tracing lines on a map by hand.
Motivation: The network has many lines and interchange stations. Working out the best route manually is slow and easy to get wrong, mostly for first-time riders.
Intended users: Delhi Metro passengers (everyday users, first-time users and tourists, and people planning a trip).
Main goal: The user picks a source and a destination. The application shows the fastest route, the metro lines, the interchanges and the estimated travel time, and highlights the route on an interactive map.

3 Final System

The final application covers **254 stations and 12 line entries**: ten lines, where the Blue and the Green line each also have their branch stored as a separate entry. The user picks a source and a destination by typing a station name or code, or by clicking on the map, and presses *Find route*. The route panel then gives the total time, the number of stops and interchanges, and step-by-step instructions, and the same route is highlighted on a zoomable and pannable map drawn over an OpenStreetMap background (Figure 1). The shortest possible interaction is three actions: pick source, pick destination, press Find route.

Not planned in A1–A3 but added: a swap button, a list of all stations A to Z when a box is empty, a From/To card when a station is clicked on the map, and a map background that is stored with the application (no tile server is contacted).

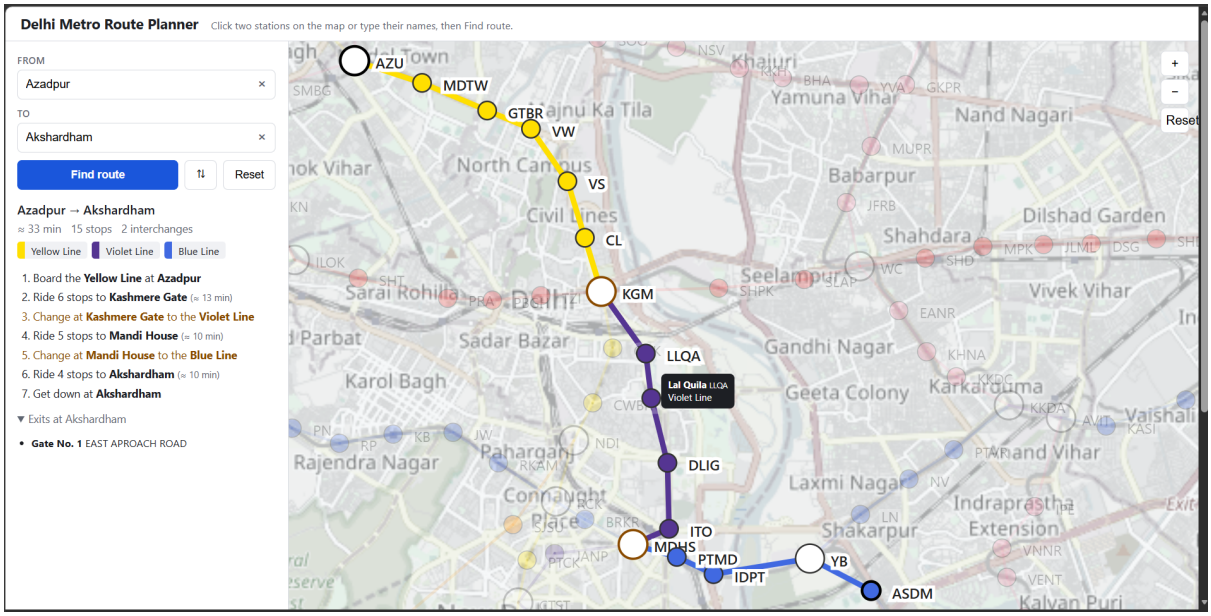


Figure 1: The running application for Azadpur → Akshardham: about 33 min, 15 stops, 2 interchanges (Yellow, Violet and Blue Line). Left: search boxes, summary, steps and the gate list of the destination. Right: the route is drawn thick and the rest of the network is dimmed. Change stations (Kashmere Gate, Mandi House, Yamuna Bank) have a brown ring, source and destination a black ring. The dark card is the hover card of Lal Quila (LLQA).

3.1 Requirement coverage

ID	Status	What was built
FR1 Station search	Done	Substring match on name and code, ranked (exact code, prefix, substring); case and extra spaces ignored; 150 ms debounce; at most 8 results; arrow keys, Enter and Esc work.
FR2 Select source and destination	Done	From the suggestions or by clicking the map (a card offers “From here / To here”). Find route stays disabled until two different stations are chosen, and its tooltip says what is missing.
FR3 Fastest route	Done, changed	Dijkstra on ride minutes. The interchange penalty was dropped (Section 8).
FR4 Interchange detection	Done	Taken from the line stored on each edge; the count equals the number of line changes.
FR5 Journey details	Partly	Summary (time, stops, interchanges), line chips and steps (board, ride, change, get down). Gates are listed for the destination only, not “which exit for which place”.
FR6 Display map	Done	Stations at dataset coordinates, edges coloured by line, interchanges drawn bigger with a thick ring, legend; drag to pan, wheel, pinch or +/− to zoom.
FR7 Highlight the journey	Done	Route drawn thick, all other stations and lines dimmed; source and destination ringed in black, change stations in brown.

ID	Status	What was built
FR8 Station hover	Partly	Name, code, lines, interchange flag. Exits and nearby transport are not shown on hover.
FR9 Clear journey	Done	Reset clears the boxes, route and highlight and returns the map to its default view; Swap exchanges the two stations.
FR10 New data	Partly	New stations and lines need only a data change, but the server must be restarted.

4 Architecture

We kept the four layers of A2 (Figure 2). Dependencies point downwards only: `routing.py` never imports `api`, nothing in `metro/` knows that HTTP exists, and the frontend knows nothing about NetworkX. The four modules asked for in the project brief map as: *metro data* = data/model layer, *route finding* = algorithm layer, *user interface* = `search.js`, `panel.js`, `state.js`, `main.js`, and *map visualisation* = `map.js`.

Start-up and data flow. When the server starts it loads and validates `metro.json` once and builds the graph and the station dictionary. The browser then fetches the whole network once (`/api/network`) and draws it once; afterwards it only asks the server for searches and routes. One process serves both the API and the built frontend.

Figure 3 shows one route request. Every request gets an id in `state.js`; if the user changes a station or presses Reset before the reply arrives, the old reply is dropped and never drawn.

Technology stack. Same as A2; only the testing and checking tools were added.

Component	Technology	Reason
UI and map	JavaScript, Vite, plain SVG	Small single-page app; a route is highlighted by restyling existing SVG elements.
Backend	Python 3.11+, FastAPI, Uvicorn	One Python codebase for the API and the algorithms.
Shortest path	NetworkX	Well-tested Dijkstra, so we test the modelling, not a rewrite.
Data storage	Static JSON file	Data is fixed and small.
Testing and checks	pytest, pytest-cov, vitest with jsdom, ruff, radon	Unit, integration and coverage on both sides.
Version control, CI	Git, GitHub Actions	Tests, ruff and build run on every push.

5 Data and Graph

5.1 Dataset

The network is one file, `backend/data/metro.json`, with the arrays *lines* (id, name, colour), *stations* (id, name, code, lat, lon, lines, optional exits) and *edges* (a, b, line, minutes, time_source).

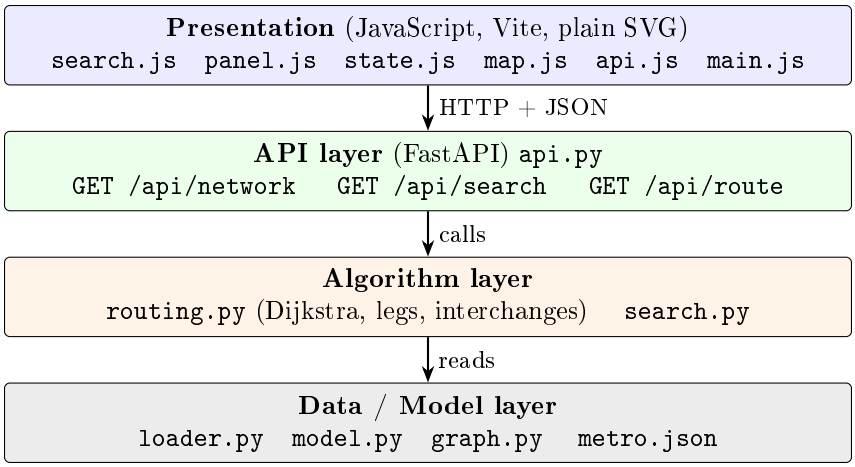


Figure 2: Layered architecture. The graph and the station dictionary are built once from `metro.json` when the server starts and are read-only afterwards.

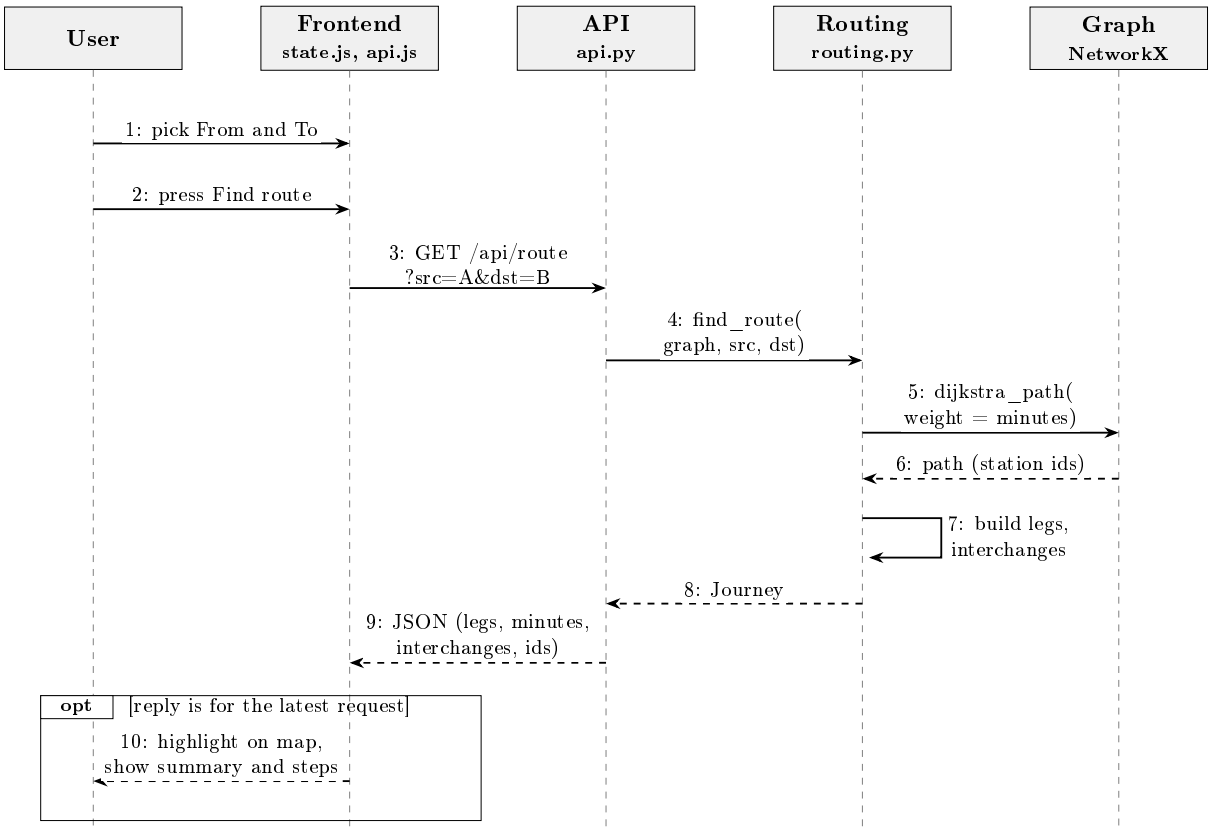


Figure 3: Sequence diagram of one route request. Solid arrows are calls, dashed arrows are returns.



Figure 4: The 254 stations and 12 lines in `metro.json`. Interchange stations have a white ring.

One edge is one pair of adjacent stations on one line. Current size: **12 lines, 254 stations, 273 edges, 29 interchange stations**. A script, `build_metro.py`, builds the file offline from:

- **OpenStreetMap**: stations, order of stations on each line, coordinates, line colours.
- **DMRC static GTFS timetable (Aug 2023)**: official ride times for 253 of the 273 edges, and station codes.
- **DMRC station and gate sheet**: 585 gates at 211 stations (the exits shown in the panel).

The other 20 edges opened after that timetable. Their time is estimated from distance, using a speed fitted on the timetabled edges, and they are marked `"time_source": "estimate"`. Raw DMRC files are not committed (their terms of use), only the derived JSON. Figure 4 is drawn from this file.

When the server starts, `loader.validate()` checks the rules the graph relies on before it is built: unique ids and codes, every edge end exists, the edge's line is listed at both stations, `minutes` > 0 (Dijkstra needs non-negative weights), every station has a connection, coordinates in range. All problems are listed in one pass so the data file can be fixed at once.

5.2 Graph representation

As planned in A1 and A2, the network is a weighted undirected graph stored as an adjacency list (NetworkX `Graph`, a dictionary of neighbour dictionaries):

$$G = (V, E), \qquad V = \text{station ids}, \qquad w(a, b) = \text{ride minutes}$$
$$E = \{\{a, b\} : \text{a line runs directly between } a \text{ and } b\}$$

Each edge also stores the *set* of lines that run on it. Two data decisions mattered:

- **Branches are separate line ids** (Blue branch to Vaishali, Green branch to Ashok Park Main), because the rider really changes trains at Yamuna Bank or Kirti Nagar. Interchange detection then needs no special case.

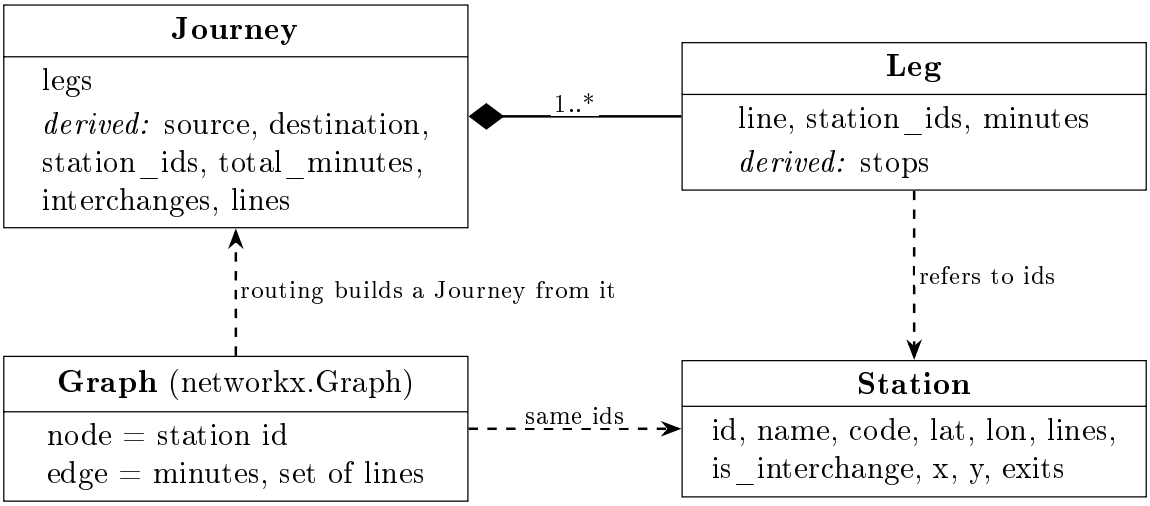


Figure 5: Class diagram of the data structures. `Journey` and `Leg` are returned by routing; `Station` is the one place where a station's name, code and map position live.

- **Shared track.** Between Anand Vihar and Karkarduma the Blue branch and the Pink Line run on the same track in the same time. The graph keeps one edge with both lines in its set, so a Pink Line rider is not told to change to Blue for one stop and back. (This is why the graph has 272 edges for 273 data rows.)

The other data structures of A2 are shown in Figure 5. A `Journey` stores only its legs; total time, interchanges, lines and the station ids for the map are computed from them, so they cannot disagree with the legs. The priority queue is inside NetworkX's Dijkstra.

6 Algorithms

Shortest route (Dijkstra). We call `nx.dijkstra_path` with `minutes` as the weight, as decided in A2. The validator guarantees positive weights, so Dijkstra is correct. BFS would minimise the number of stops, not minutes, and A* gives nothing on 254 stations.

Leg reconstruction and interchanges. Dijkstra returns only station ids. The path is then walked edge by edge:

1. Stay on the current line while the next edge also has that line.
2. When it does not, the station between the two edges is an interchange: close the leg and start a new one there.
3. If an edge has several lines (shared track), pick the line that also runs on the next edge, so a leg is not split for nothing.

For Azadpur → Akshardham this gives three legs: Yellow, 6 stops, 12.6 min → Violet, 5 stops, 10.4 min → Blue, 4 stops, 9.8 min; total 32.8 min and 2 interchanges (Kashmere Gate, Mandi House), the same numbers as in Figure 1. This differs from A2 only in that line changes are read from the edges and not from the stations (Section 8).

Station search. The text is trimmed, lower-cased and cut at 50 characters, then each station is tested with a plain substring check on name and code (no regex). Results are ranked exact code, prefix, substring, and at most 8 are returned.

Complexity and measured performance. $V = 254$, $E = 273$. Times were measured in-process (no HTTP) with Python 3.13 on every one of the $254 \times 253 = 64{,}262$ ordered station pairs; $L \leq V$ is the number of stations on the route.

Step	Time	Space	Measured
Load, validate, build graph	$O(V + E)$	$O(V + E)$	15 ms, once at start-up
Dijkstra (binary heap)	$O((V + E) \log V)$	$O(V)$	Whole `find_route`: mean 0.24 ms, 95th percentile 0.56 ms, worst 6.8 ms
Leg reconstruction	$O(L)$	$O(L)$	Included above
Station search	$O(N \cdot L_{name})$	$O(1)$	About 0.1 ms per query

The A3 target was about 100 ms per route and no memory growth after 1000 requests. Both hold: the worst route takes under 7 ms, and the memory reported by `tracemalloc` was the same (6 KB) after 1000 and after 2000 routes. The API test suite also asserts a 95th percentile under 100 ms for `/api/route`. Zoom and pan only change the SVG `viewBox`; the whole network (273 edge lines, 254 station markers and 64 map tiles) is drawn once.

7 Testing and Results

We followed the A3 plan: black-box techniques (equivalence classes, boundary values, decision table, state transitions) plus coverage, with tests written together with each component, and test cases in the [I, S, O] form.

Side	Tool	Tests	Coverage	Mutants
Backend	pytest, pytest-cov	166	100% statement and branch on every module	37/37 killed
Frontend	vitest, jsdom	89	99.7% lines, 93% branches (100% lines on `api, state, panel, main, search`; 99% on `map`)	14/14 killed

What is covered.

- **Loader and graph:** one fault per test (duplicate id, unknown line, zero or negative minutes, ...) with boundaries for coordinates and minutes; interchange flags; shared track keeps both lines.
- **Routing:** a hand-checked fixture (9 stations, 4 lines, 2 interchanges, one separate line) for neighbouring stations, same-line, one-change and "faster path with a change beats the slower direct line" cases; on more than 400 real station pairs the total is compared with `nx.dijkstra_path_length`, legs are checked to be consecutive and on the right line, and the result is the same in both directions.
- **Search:** part of a name, full name, code, capitals, spaces, empty, very long text.
- **API and integration:** contract of the three endpoints, a 20-thread test of the shared read-only state, and a test that a station added only in the JSON appears in search, routing and the map (FR10).
- **Frontend:** the store (including out-of-order replies), panel text, search box, map pan/zoom/highlight, and a jsdom boot of the real page.

Regression. One command per side runs everything, and GitHub Actions runs both on every push and pull request. The backend job fails if coverage drops below 100%, and also runs ruff and radon (no function above complexity 10; the highest is 9).

Verification and validation. The tests above are verification against A1. For validation we compared Rajiv Chowk → Kashmere Gate with the official DMRC planner: 8.72 min against 8:37 (about 1% apart). That is one journey only; more comparisons would make the claim stronger. The usability session with classmates planned in A3 was not run, so we make no claim about the three-action target beyond the design itself.

8 Deviations from A1, A2 and A3

Topic	Planned	Final
Interchange penalty (A1 FR3, A3 R2)	Cost = travel time + penalty per interchange.	Dropped: plain Dijkstra on ride minutes. We have no measured transfer times, and the penalty needs a (station, line) graph, as A3 R2 warned. Cost: Samaypur Badli → Noida City Centre uses 2 changes (62.6 min) where 1 change at Rajiv Chowk takes 65.4 min.
Fewest-interchange route (A1 3.1)	Offered as a second mode.	Not offered; needs the same (station, line) graph.
Interchange detection (A2 Alg. 2)	Compare the lines of consecutive stations.	Compare the line sets stored on each edge; this handles branches and shared track.
Branches (new)	Not planned.	Blue and Green branches are separate line ids.
Exit details (A1 FR5)	Which exit for which destination.	Gate list of the destination station, from the DMRC sheet (211 of 254 stations have data).
Hover (A1 FR8, A3 5.1)	Lines, exits, nearby transport.	Name, code, lines, interchange flag.
Small-screen layout (A3 5.1)	Panel below the map.	Panel above the map.
Data reload (A1 FR10)	New data appears without code change.	Same, but with a server restart (A3 L3).
Network (A1)	All Delhi Metro lines.	Aqua Line (Noida, other operator, joined by a walkway between two different stations) left out.
Ideas from A3	Hindi labels, alternative routes, fares.	Not built.

9 Challenges and Lessons

- **Data was harder than the algorithm.** Dijkstra is one library call; a clean dataset (stations, order on each line, times, codes, gates) from three sources took much longer. The validator and the `time_source` marker helped us trust it.
- **An interchange belongs to the line, not to the station.** Comparing the lines of consecutive stations fails for branches and shared track; storing the set of lines on the edge

fixed it.

- **A3 risk R2 was real.** One node per station cannot price a line change. We chose to say so instead of inventing a constant.
- **Tests and layers.** Writing tests with each component caught small bugs early (validation, leg splitting), and the separate layers kept changes cheap: adding gate data touched the data builder and one panel function. Out-of-order replies needed a request id and a test.

10 Limitations and Future Work

Limitations.

- No live information (delays, closures), no peak and off-peak difference, no waiting or walking time at interchanges.
- No fares, buses or last-mile routes; one route type only (fastest).
- Edges are undirected, so a ride takes the same time both ways.
- The Magenta Line is two separate parts in the data because one stretch is not open yet.
- Exit data depends on the DMRC sheet; the interface is in English only.

Future work. A (station, line) graph, which unlocks both an interchange penalty and a fewest-interchange mode; alternative routes with comparison; fares; Hindi labels; reloading data without a restart; and a proper usability study.